Ders 6 Genişletilmiş Notu: Kayan Nokta Hataları, Veri Türleri, ASCII/UTF-8 ve Veri Sıkıştırmaya Giriş

Ders 6 Genişletilmiş Notu: Kayan Nokta Hataları, Veri Türleri, ASCII/UTF-8 ve Veri Sıkıştırmaya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Derse Giriş ve Kayan Nokta Gösteriminin Tekrarı
- 2. Birinci Örnek: 2 Tam 3/4
- 3. İkinci Örnek: 3/8 ve Mantisin İlk 1'den Başlaması
- 4. Kesme Hatası (Truncation Error)
- 5. Single ve Double Precision
- 6. Farklı Sistemler, Farklı Kurallar
- 7. Veri, Bilgi ve Bilgisayarda Saklanan Veri Türleri
- 8. Karakterlerin Kodlanması: ASCII Tablosu
- 9. UTF-8'e Neden İhtiyaç Duyuyoruz?
- 10. Bir Sayıyı Sayı ve Karakter Olarak Saklamak
- 11. Resim ve Video
- 12. Ses, Diziler ve Değişkenler
- 13. Veri Sıkıştırma

1. Derse Giriş ve Kayan Nokta Gösteriminin Tekrarı

Hoca derse, önümüzdeki dönemde çözülecek problemlerin nasıl ele alınacağını anlatarak başladı. Problemler sayı sistemleri, bilgisayarın eksi sayıyı nasıl ifade ettiği ya da algoritmalar gibi konulardan gelebilir. Önemli olan şudur: Bir problemin tek bir çözümü yoktur. Bu yüzden birden fazla çözüm konuşulacak ve karşılaştırılacaktır. Hangisi daha iyidir, neden daha iyidir, ne farklı yapılsaydı problem daha efektif, bilgisayara daha az işlem yaptırılarak çözülürdü? Sorulacak sorular bunlardır.

Ardından geçen haftanın konusuna dönüldü: kesirli sayıların gösterimi ve bellekte saklanması. Bellekteki bit grubunda üç bölge vardır:

- **İşaret biti (sign bit):** Sayının artı mı eksi mi olduğunu belirler.
- **Üs alanı (exponent field):** Noktanın sağa ya da sola ne kadar kayacağını gösterir. Nokta normalde yalnızca kağıt üzerinde gösterilir.
- **Mantis alanı (mantissa field):** Sayının ikilik sistemdeki hem tam kısmının hem kesirli kısmının bit karşılığıdır. Nokta bu blok üzerinde kaydırılır.

Dersteki örneklerin hepsi 8 bitlik biçimdedir: 1 bit işaret, 3 bit üs, 4 bit mantis. Üs alanı excess-4 gösterimiyle yazılır, yani alandaki değerden 4 çıkarılınca gerçek üs bulunur:

$$\text{üs} = (\text{üs alanının değeri}) - 4$$

Bir sayıyı bu biçime çevirirken izlenecek adımlar örneklerden şöyle çıkar:

1. İşaret bitini yaz: Sayı pozitifse 0, negatifse 1.

2. Sayıyı kağıt üzerinde yazıldığı gibi, noktalı ikilik biçimde yaz.
3. Bu yazımı ilk 1'den başlayarak mantis alanına yerleştir.
4. Noktanın, mantisin başındaki konumundan gerçek konumuna kaç basamak ve hangi yöne kaydığını bul. Bu değeri excess-4 ile üs alanına yaz.

Sağlama aynı yolun tersidir: İşaret okunur, mantis yazılıp başına nokta konur, üs alanı çözülür ve nokta o kadar kaydırılır.

2. Birinci Örnek: 2 Tam 3/4

Hoca $2\frac{3}{4}$ sayısının 0 ve 1 kombinasyonlarıyla nasıl gösterileceğini sordu. Bir öğrenci çözümün sonucunu söylemeye başlayınca hoca sonucu değil, adımları istediğini belirtti. Başka bir öğrenci tam kısmı 2'ye bölmeyi önerdi. Hoca tam kısım ile $\frac{2}{4}$ ve $\frac{1}{4}$ kesirlerini ayrı ayrı göstererek bunun doğru olup olmadığını sordu. Cevaplar net gelmeyince çözümü kendisi adım adım kurdu.

> **Hocanın vurgusu:** Verilen cevaplardan, derslerden sonra çözümlere yeterince vakit ayrılmadığı anlaşılıyor. Her ders sonrasında örnekleri kendiniz tekrar çözmelisiniz.

Verilen: $2\frac{3}{4}$, 8 bitlik biçim.

Adım 1, işaret: Bir öğrenci işaret bitinin negatif olup olmadığını sordu. Hoca, sayının herhangi bir yerinde eksi işareti olmadığını, bu yüzden işaret bitinin 0 olduğunu söyledi.

Adım 2, normal yazım: Sayı kağıt üzerinde yazıldığı gibi yazılır. Nokta, tam kısım ile kesirli kısım arasına girer:

$$2\frac{3}{4} = 10.11_2$$

Adım 3, mantis: Bu grup olduğu gibi mantis alanına yazılır: `1011`. Grup zaten 1 ile başladığı için en baştan alınır.

Adım 4, üs: Nokta, mantis alanında normalde en başta durur. Gerçek konumuna gelmesi için iki basamak sağa kayması gerekir, yani üs $+2$'dir. Bir öğrencinin de söylediği gibi excess-4'te $+2$'nin karşılığı 110'dur, çünkü $110_2 = 6$ ve $6 - 4 = 2$.

Sonuç: `0 | 110 | 1011`. Bu 8 bit, $2\frac{3}{4}$ yani 2.75 sayısını gösterir.

2.1 Sağlama

Bir öğrenci sağlamayı akıldan yaptı: 110 iki birim kayma demektir, noktadan sonraki 11 de $\frac{1}{2} + \frac{1}{4} = \frac{3}{4}$ eder. Hoca bunun yanlış olmadığını, ama alışana kadar adım adım gidilmesi gerektiğini söyledi ve sağlamayı elinde yalnızca bit grubu varmış gibi yeniden yaptı:

1. İlk bit 0, demek ki sayı artıdır.
2. Mantis yazılır ve başına nokta konur: $.1011$
3. Üs alanı 110 excess-4'te $+2$'dir, nokta iki basamak sağa kayar: 10.11
4. Basamak değerleri $2^1, 2^0, 2^{-1}, 2^{-2}$'dir:

$$1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 2 + \frac{1}{2} + \frac{1}{4} = 2\frac{3}{4}$$

Soru: 2^0 neden toplama işlemine eklenmedi? **Cevap:** Çünkü 2^0 basamağının karşısında 0 yazıyor. $0 \cdot 2^0 = 0$'dır.

Soru: En küçük excess gösterimi nedir? **Cevap:** "En küçük excess gösterimi" diye bir ifade yoktur. İsterseniz excess-2 de alabilirsiniz. O zaman ifade edebileceğiniz aralık değişir.

3. İkinci Örnek: 3/8 ve Mantisin İlk 1'den Başlaması

Hoca ikinci örnek olarak $\frac{3}{8}$ sayısını verdi. Bu örnekte öğrenciler birkaç yanlış deneme yaptı. Hoca her denemeyi hesaplatarak hatanın nerede olduğunu gösterdi.

3.1 Yanlış Denemeler

Birinci deneme: Bir öğrenci işaret bitini 0 yaptıktan sonra sayıyı 0.110 olarak yazmayı önerdi. Hoca hesabı birlikte yaptırdı:

$$0.110_2 = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}$$

Bu $\frac{3}{8}$ değil, $\frac{3}{4}$'tür. Deneme bu yüzden silindi.

İkinci deneme: Başka bir öğrenci doğrudan "mantis 1100 olsun, nokta en başta dursun" dedi. Hoca burada bir kez daha sırayı hatırlattı.

> **Hocanın vurgusu:** Kendi kafanıza göre gitmeyin, adım adım gidin. Önce işaret biti, sonra mantis, sonra üs alanı. Yanlış yerden başlıyorsunuz: Önce yazmanız gereken sayı, kağıt üzerindeki noktalı gösterimiyle $\frac{3}{8}$'dir.

Hoca, bu gruptaki 1100'ün karşılığını sorunca öğrenci 12 dedi ve hoca bunu onayladı.

> **Ders sonrası not:** Hoca 1100 için "12'ye denk geliyor" dedi. 12, bu dört bitin tam sayı olarak okunmuş değeridir. Nokta en baştaysa değer $.1100_2 = \frac{12}{16} = \frac{3}{4}$ olur. Her iki okumada da sonuç $\frac{3}{8}$ değildir.

3.2 Doğru Çözüm

Verilen: $\frac{3}{8}$, 8 bitlik biçim.

Adım 1, işaret: Sayı pozitif, işaret biti 0.

Adım 2, normal yazım: Bir öğrenci $.0110$ yazdı. Hoca basamak değerlerini $2^{-1}, 2^{-2}, 2^{-3}, 2^{-4}$ olarak işaretleyip kontrol etti:

$$.0110_2 = 2^{-2} + 2^{-3} = \frac{1}{4} + \frac{1}{8} = \frac{3}{8}$$

Sayı artık gerçekten kağıt üzerindeki biçimiyle yazılmıştır.

Adım 3, mantis: Öğrenci mantise de 0110 yazmak istedi, fakat kural şudur: Mantise ilk 1'den itibaren yazılır. Baştaki 0 atılır, 110 alınır. Mantis 4 bit olduğu için kalan yer 0 ile doldurulur. Hoca bunun nedenini şöyle açıkladı: Sayıyı daha uzun yazsaydınız sağ tarafı zaten 0 olarak devam edecekti. Sonuç: `1100`.

Adım 4, üs: Mantisin başındaki noktanın gerçek konuma gelmesi için bir basamak **sola** kayması gerekir, yani üs -1'dir. Excess-4'te bu $-1 + 4 = 3$ yani 011 olur. Sağlaması: $011_2 = 3$ ve $3 - 4 = -1$.

Sonuç: `0 | 011 | 1100`.

> **Hocanın vurgusu:** Mantis yazılırken grubun başında 0 varsa mutlaka 1 olan gözden itibaren alınır.

3.3 "Bu da Doğru Olabilir" Tartışması ve Sağlama

Hoca sınıfa bu grubun doğru olmadığını düşünen var mı diye sordu. Bir öğrenci "bu da doğru olabilir, ama ben şöyle yaptım" diye başlayınca hoca şu benzetmeyi yaptı: Matematikte $2 + 2$ 5 de olabilir, 6 da olabilir diye bir şey yoktur. Bir çözüm ya doğrudur ya yanlıştır. Asıl soru şuydu: Ekrandaki sıfır ve birlerden oluşan grup $\frac{3}{8}$'i ifade ediyor mu, etmiyor mu?

Bazı öğrenciler grubun $\frac{3}{8}$'i ifade etmediğini, ya da üs alanı sıfırdan küçük olduğu için böyle kaydırılamayacağını söyledi. Hoca tartışmayı sağlama ile çözdü:

1. İlk bit 0, sayı artıdır.
2. Mantis yazılır, başına nokta konur: $.1100$
3. Üs alanı 011 excess-4'te -1'dir. Başında eksi olduğu için nokta bir basamak sola kayar.
4. Noktanın boşalttığı yere 0 gelir: $.01100$
5. Değer: $2^{-2} + 2^{-3} = \frac{1}{4} + \frac{1}{8} = \frac{3}{8}$

Demek ki grup $+\frac{3}{8}$'i gösterir ve çözüm doğrudur.

Hocanın burada altını çizdiği nokta şudur: Noktayı sola kaydırdığınızda sayıyı kaydırmıyorsunuz, yalnızca noktayı kaydırıyorsunuz. Birler ve sıfırlar yerinde kalır, noktanın boşalttığı basamağa otomatik olarak 0 gelir.

> **Hocanın vurgusu:** Sayıyı oluşturduktan sonra sağlamasını yapın. Geri dönüşte aynı sayıyı elde ediyorsanız yaptığınız iş doğrudur.
>
> **Ek bilgi (derste anlatılmadı):** Mantisin her zaman ilk 1'den başlaması, aynı sayının tek bir biçimde yazılmasını sağlar. Gerçek bilgisayarlardaki standart büyük olasılıkla IEEE 754'tür. Bu standartta mantis büyük olasılıkla $.1xxx$ değil $1.xxx$ biçiminde düşünülür ve baştaki 1 yazılmadan saklanır. Üs alanındaki kaydırma değeri de büyük olasılıkla 32 bitte 127, 64 bitte 1023'tür. Dersteki 8 bitlik biçim kuralı anlatmak içindir, bu ayrıntılarla birebir aynı değildir.

4. Kesme Hatası (Truncation Error)

4.1 1/3 ve 1/20 Gibi Kesirler

Soru: Bu kayan nokta gösterimiyle $\frac{1}{3}$ ya da $\frac{1}{20}$ gibi kesirleri bulmamız mümkün mü? **Cevap:** Hoca önce örneklerin hep 2'nin kuvvetleri üzerinden gittiğini hatırlattı. Bir öğrenci, mantis yeterince geniş olmadığı için değer kaybı olacağını ve buna truncation error dendiğini söyledi. Hoca bunu onayladı: Mantis kısıtlı olduğu için 8 bitlik alanda bu kesirlerin gerektirdiği ayrıntıyı ifade etme şansımız yoktur. Daha hassas kesirli sayılar için mutlaka daha geniş bir alan gerekir. Bu özellikle mantis için geçerlidir, ama üs alanı da büyür. 8 bit yerine 32 bit ya da 64 bit kullanıldığında bu alanların büyüdüğü görülür. Yalnızca işaret biti değişmez, kalan bitler üs ve mantis arasında paylaşılır. Bilgisayarlar da zaten böyle çalışır.

> **Ders sonrası not:** Hoca, alan genişleyince bu kesirlerin rahatça ifade edilebileceğini söyledi. Doğrusu, $\frac{1}{3}$ ve $\frac{1}{20}$ ikilikte sonsuza kadar tekrar eder: $\frac{1}{3} = 0.\overline{01}_2$, $\frac{1}{20} = 0.00\overline{0011}_2$. Bu yüzden hiçbir sonlu mantisle tam ifade edilemezler. Alan genişleyince yalnızca hata küçülür. Python ile 32 bitte saklanan değerler $0.3333333432\ldots$ ve $0.0500000007\ldots$ çıkıyor.

4.2 Örnek: 2 Tam 5/8

Hoca bu problemi $2\frac{5}{8}$ üzerinden gösterdi. Bu sayı, şimdiye kadar yazılanlara göre 8 bite oranla daha yüksek hassasiyet ister. Hoca, sınıfı denemeye teşvik ederken hata yapmakta bir sakınca olmadığını, zamanla alışılacağını da söyledi.

Bir öğrenci işaret bitini 0 yaptıktan sonra sayıyı önce 1.101 olarak yazdı. Hoca basamak değerlerini yazarak hesaplattı:

$$1.101_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1\frac{5}{8}$$

Bu istenen sayı değildir, tam kısım 2 olmalıdır. Düzeltilmiş çözüm şöyledir:

Verilen: $2\frac{5}{8}$, 8 bitlik biçim.

Adım 1, işaret: Sayı pozitif, işaret biti 0.

Adım 2, normal yazım: Nokta ilk sıfırın sağındadır. Basamak değerleri $2^1, 2^0, 2^{-1}, 2^{-2}, 2^{-3}$'tür:

$$10.101_2 = 2 + \frac{1}{2} + \frac{1}{8} = 2\frac{5}{8}$$

Adım 3, mantis: Öğrenci önce üs alanına geçmek istedi, ama hoca önce mantisin yazılmasını istedi. Grubu olduğu gibi yerleştiremeyiz: Mantiste 4 bit yer vardır, bizde 5 bit vardır. İlk 1'den itibaren 4 bit alınır: `1010`. Son bit sığmaz.

Adım 4, üs: Nokta iki basamak sağa kayacak, üs $+2$, alan 110 olur ($6 - 4 = 2$).

Sonuç: `0 | 110 | 1010`.

Sağlama: Sayı artıdır. Mantis yazılıp başına nokta konur: $.1010$. Üs $+2$ olduğu için nokta iki sağa kayar: 10.10. Değer:

$$10.10_2 = 2 + \frac{1}{2} = 2\frac{1}{2} = 2\frac{4}{8}$$

4.3 Ne Oldu?

$2\frac{5}{8}$ yazmak istedik ama $2\frac{4}{8}$ yazabildik. Kaybolan bit $\frac{1}{8}$ bitidir. Sebebi, sayının 5 bit hassasiyet gerektirmesi, mantis alanının ise 4 bit olmasıdır. En sondaki bit düşmüştür. Bilgisayar ya da elektronik cihaz, kullanıcının verdiği $2\frac{5}{8}$ değerini işleme $2\frac{4}{8}$ olarak sokar.

Buna **truncation error** denir. Hocanın verdiği diğer adıyla **round-off error**, alanın dar olmasından kaynaklanır.

> **Hocanın vurgusu:** Truncation error bilgisayara hata verdirmez. Yalnızca oradaki hassasiyeti kaybederek işlemlere devam edersiniz. Günün sonunda iki bilgisayarın sonucunu karşılaştırdığınızda birinde küsuratın farklı olduğunu görebilirsiniz. Sebep, işleme giren sayı arka planda saklanırken bu bitin otomatik olarak düşmesidir.

Bu bölümün sonunda sohbetten gelen bir yazım sorusu da geçti. Hoca, arkadaşlarının yazdığı iki nokta ile ayrılan kısımların büyük olasılıkla işaret biti ve üs alanını ayırdığını tahmin ettiğini söyledi.

5. Single ve Double Precision

Hoca, 8 bitlik örnekten gerçek bilgisayarlarda kullanılan biçimlere geçti. İki yaklaşım vardır: **single precision floating point** ve **double precision floating point**. Aralarındaki fark, bir öğrencinin de söylediği gibi bit sayısıyla, dolayısıyla hassasiyetle ilgilidir.

- **Single precision:** 32 bitlik gösterimdir. Az önceki 8 bitlik yapı 32 bitle ifade edilir.
- **Double precision:** 64 bitlik alan kullanır. Hem kullanılabilecek sayı aralığı hem sayı hassasiyeti oldukça genişler.

Hoca 32 bitin nasıl paylaşıldığını sordu. Bir öğrencinin "1, 15, 16" önerisinin ardından doğrusunu verdi:

$$32 = 1\ (\text{işaret}) + 8\ (\text{üs}) + 23\ (\text{mantis})$$

Hoca bunu ezbere bilmenizi beklemediğini, bilinmesi gereken tek kısmın işaret için 1 bit olduğunu söyledi.

Bu biçimin özellikleri derste şöyle verildi:

- **Single precision (32 bit):** Yaklaşık 10^{-37} ile 10^{38} arasını ifade eder. Hoca hassasiyeti noktadan sonra yaklaşık 7 dijit olarak verdi.
- **Double precision (64 bit):** Hoca hassasiyeti noktadan sonra 15 ondalık dijite (decimal digit) kadar olarak verdi.

> **Ders sonrası not:** Hoca hassasiyeti "noktadan sonra" 7 ve 15 dijit olarak söyledi. Doğrusu büyük olasılıkla toplam anlamlı basamak sayısıdır, çünkü nokta kaydıkça noktadan sonraki basamak sayısı değişir. 00:42:26'daki "15 bitlik" ifadesi de 15 dijit olmalıdır. Aralığın alt sınırı da yaklaşıktır: Python ile tek duyarlıkta en büyük değer yaklaşık 3.40×10^{38}, standart biçimdeki en küçük pozitif değer yaklaşık 1.18×10^{-38} çıkıyor.

> **Ders sonrası not:** Hoca, "32 bitlik bir bilgisayarda" 1 + 8 + 23 bit kullanıldığını söyledi. Bu paylaşım büyük olasılıkla bilgisayarın 32 bitlik olmasına bağlı değildir, sayı tipinin 32 bitlik olmasına bağlıdır. Bu makinede `gcc` ile `sizeof(float)` 4, `sizeof(double)` 8 byte çıkıyor.

> **Ek bilgi (derste anlatılmadı):** Derste double precision için yalnızca toplam bit ve hassasiyet verildi. 64 bitin paylaşımı büyük olasılıkla 1 bit işaret, 11 bit üs ve 52 bit mantistir. İşaret bitinin yine 1 bit kalması, hocanın "yalnızca işaret biti değişmez" sözüyle uyumludur.

6. Farklı Sistemler, Farklı Kurallar

Aradan sonra hoca kısa bir toparlama yaptı. Bilgisayarın tam sayıları, negatif sayıları ve kesirli sayıları nasıl sakladığı kabaca görülmüştür. Bu gösterim, günlük hayattaki matematiksel gösterimden çok farklıdır, çünkü bellekte eksi ya da nokta gibi işaretlerle bilgi saklanamaz. Onların da bir bit karşılığı, yani 0 ve 1 kombinasyonu vardır.

Bir bilgi komut, resim ya da harf olabilir. Hepsi her zaman 0 ve 1 kombinasyonu olarak saklanır, ama bunun kuralları vardır. Her ortamda aynı kural kullanılmayabilir. Her işlemci mimarisinde ikiye tümleyen (two's complement) ya da excess gösterimi kullanılmıyor olabilir. Farklı gösterimler vardır, burada görülenler yaygın olanlardır.

Soru: Kesirli sayıların işaretsiz (unsigned) hali var mı? **Cevap:** C dilinde `float` ve `double` için işaretsiz tanım yapamıyoruz. Hocanın bildiği kadarıyla diğer programlama dillerinde de kesirli sayıların işaretsiz karşılığı yoktur. Bu yüzden kesirli sayılar genellikle eksi ve artı aralığında kullanılır.

7. Veri, Bilgi ve Bilgisayarda Saklanan Veri Türleri

7.1 Veri ve Bilgi

Hoca veri (data) ile bilgi (information) kavramlarının birbirinden farklı olduğunu söyledi. Veri anlamlı hale geldiği zaman bilgi olarak adlandırılır.

> **Hocanın vurgusu:** Her veri bilgi değildir, ama her bilgi veri içerir. Elinizdeki her veriyi bilgi olarak anlamlandıramayabilirsiniz. Verinin anlamlandırılmış haline bilgi diyoruz.

7.2 Hangi Veriler Saklanır?

Hoca sınıfla birlikte, 0 ve 1 kombinasyonlarıyla saklanan verilerin listesini çıkardı:

- Sayılar: pozitif, negatif ve kesirli sayılar.
- Harfler (letters).
- Renkler (colors).
- Komutlar: CPU komutları (CPU instructions).
- Sesler.
- Resimler (image) ve videolar.
- Karakterler: noktalama işaretleri, enter tuşu, boşluk (space) tuşu ve rakamlar bu başlığa girer. Hoca "harfler" başlığı yerine aslında "karakter seti" demek gerektiğini söyledi. Bunun altında ASCII ve UTF-8 vardır.
- Dosyalar (files): metin, resim ya da ses dosyası olabilir. Bunlar hep bu başlık altında saklanır.
- Web sayfaları ve programlar. Bütün programlar da 0 ve 1 kombinasyonlarıyla saklanır.

Hoca, bir öğrencinin önerdiği “şekiller, boyutlar” gibi başlıkların da olabileceğini, ama daha belirli adlar beklediğini belirtti. Listeye daha alt başlıklar eklenebilir, bazı başlıklar gruplanabilir. Asıl mesele, dış dünyadaki bu bilgilerin bir şekilde kodlanarak bilgisayara aktarılmasıdır.

8. Karakterlerin Kodlanması: ASCII Tablosu

8.1 Her Karaktere Bir Sayı

Sayıların nasıl kodlandığı görüldü. Sıradaki adım harflerdir. Hoca harflerin nasıl kodlanacağını sordu. Bir öğrenci her harfe bir sayı atamayı önerdi ve hoca bu fikri beğendi. Sayılarda ikiye tümleyen ve excess gösterimi gibi kurallar nasıl varsa, karakterler için de kuralları içeren tablolar kullanılır. Harflerin, noktalama işaretlerinin ve rakamların karakter bazında saklanabilmesi için hepsinin kod karşılığı vardır.

En çok bilineni ASCII tablosudur. Hoca bu tabloyu 8 bitlik ve 256 sembol saklayabilen bir tablo olarak anlattı, kodlar 0’dan 255’e gider.

> **Ders sonrası not:** Hoca ASCII’yi 8 bitlik ve 256 sembollük olarak anlattı. Doğrusu büyük olasılıkla standart ASCII’nin 7 bitlik olduğu ve 0-127 arasındaki 128 sembolü tanımladığıdır. 256 sembol, büyük olasılıkla bu tablonun 8 bite genişletilmiş biçimleriyle ilgilidir. Karakterin bellekte 1 byte’lık alanda tutulması ise dersin bundan sonraki hesaplarını değiştirmez.

Tabloda şunlar bulunur:

- **Büyük harfler:** A’dan başlayarak Latin alfabesindeki büyük harfler arka arkaya gelir.
- **Küçük harfler:** Onlar da arka arkaya gelir.
- **Rakamlar:** 0’dan 9’a kadar. Bunlar normal tam sayılardan farklı olarak karakter şeklinde saklanır.
- **Noktalama işaretleri:** nokta, virgül, ünlem, soru işareti, boşluk gibi.
- **Semboller:** örneğin et işareti.

Bu tablo tüm dünya tarafından kabul görmüştür.

> **Ders sonrası not:** Tahtadaki taslakta küçük harfler 81’den başlıyormuş gibi anlaşılabilir. Python ile gerçek değerler: `'A'` = 65, `'Z'` = 90, `'a'` = 97, `'z'` = 122. Yani büyük harfler 65-90, küçük harfler 97-122 arasındadır.

8.2 Referans Değerler

Hoca iki referans değer verdi ve kalanını sınıfa buldurdu:

- `'0'` karakterinin karşılığı 48’dir. Rakamlar sıralı olduğu için `'9'` = $48 + 9 = 57$.
- `'A'` karakterinin karşılığı 65’tir. Dolayısıyla `'B'` = 66.

> **Hocanın vurgusu:** ASCII tablosundaki değerleri hiçbir zaman ezberlememelisiniz. Problemleri yalnızca referans noktalarını kullanarak çözmelisiniz. Hep karakter sabitlerini kullanacağız, sayısal karşılıklarını ezbere hatırlamaya çalışmayacağız.

Referans noktası ile kastedilen şudur: `'1'` ile `'0'` arasındaki fark her zaman 1, `'9'` ile `'0'` arasındaki fark 9’dur. `'Z'` ile `'A'` arasındaki fark da alfabedeki harf sayısına bağlıdır. Hoca bunların ne anlama geldiğinin ileride konuşulacağını söyledi. Tablonun anlatılma sebebi de budur: Karakter sabitlerini ifade etmek için sayısal karşılıklara ihtiyaç vardır ve bunlar belirli bir kurala göre oluşturulmuş ASCII tablosunda saklanır.

> **Ders sonrası not:** Hoca `'Z'` ile `'A'` arasındaki farkın “alfabe sayısı kadar” olduğunu söyledi. Doğrusu harf sayısının bir eksiğidir: `'Z' - 'A'` = $90 - 65 = 25$. Latin alfabesindeki büyük harf sayısı 26’dır.

9. UTF-8’e Neden İhtiyaç Duyuyoruz?

Hoca, tabloda 256 sembol saklanabildiğine göre UTF-8 kodlama biçimine neden ihtiyaç duyulduğunu sordu. Bir öğrenci başka ülkelerin alfabelerini gösterdi ve hoca bunu onayladı.

Tablo ağırlıklı olarak Latin alfabesi kullanan ülkelere uygundur. Türkçe de Latin alfabesiyle büyük ölçüde kesişir, ama yumuşak g, ş, ü gibi karakterler bu tabloda yoktur. Çince ya da Kiril alfabesi gibi çok farklı karakter setleri de vardır. Özellikle uzak doğu ülkelerinde karakter setleri daha da farklı ve ifade edilmesi zordur. Bunların hepsini 256 sembol içinde ifade etmek mümkün değildir.

UTF (Unicode Transformation) formatında standart semboller ASCII ile aynı kodlarla ifade edilir. Ek semboller için kullanılan alana bağlı olarak, hocanın anlatımıyla 24 ya da 32 bitlik alan kullanılır ve böylece bütün bu dillerin bütün karakterleri rahatça ifade edilebilir.

> **Ders sonrası not:** Hoca, 24 ya da 32 bitlik alana bağlı olarak 2^{32} adet sembol ifade edilebildiğini söyledi. Doğrusu büyük olasılıkla şudur: UTF-8 değişken uzunluklu-dur, bir karakter 1 ile 4 byte arasında yer kaplar ve Unicode'un kod noktası sayısı 1.114.112 ile sınırlıdır, 2^{32} değildir. 24 ve 32 bitin yanında 8 ve 16 bitlik kodlar da vardır. Python ile ölçüm: `A` 1 byte, `ş`, `ğ` ve `ü` 2'şer byte, Kiril `Я` 2 byte, U+4E2D kodlu Çince bir karakter 3 byte. Yani hocanın saydığı Türkçe harfler 24 bit değil 16 bit kaplar.

Hoca buradan genel bir sonuç çıkardı: Daha önceki konularda olduğu gibi burada da kurallar, biçimler ve protokoller vardır. Protokoller ileride görülecektir. Örneğin iki bilgisayarın haber-leşmesi için Bluetooth, Wi-Fi ve NFC gibi farklı protokoller vardır. Bilgisayarlarda bütün işler bu protokoller üzerinden ilerler.

10. Bir Sayıyı Sayı ve Karakter Olarak Saklamak

10.1 3 ile '3' Farklıdır

Soru: Alt artı ile sayı yazdığımızda ne oluyor? **Cevap:** Hoca soruyu tam anlayamadığını söy-ledi ve konuyu şöyle bağladı: Alt artı ile yazmak gibi düşünmeyelim. Sayıları, rakamları ya da karakterleri değişkenlerde saklıyoruz. Değişken tipi sayı ise sayı biçiminde, karakter ise karakter biçiminde saklanır.

Karakter ifade ederken her zaman tek tırnak kullanılır. Tek tırnak içinde bir karakter ifade edilir: `'a'`, `'3'`, boşluk karakteri için `' '`.

3 sayısal olarak saklansaydı bellekte karşılığı şöyle olurdu:

$$3 = 00000011_2$$

Karakter olarak `'3'` saklansaydı önce tablodaki değeri bulunur. Referans noktasından sayarak: `'0'` 48, `'1'` 49, `'2'` 50, `'3'` 51'dir. Sonra 51 ikiliğe çevrilir. Hoca basamakları şöyle yazdırdı:

$$51 = 32 + 16 + 2 + 1 = 00110011_2$$

Basamaklara tek tek bakılırsa bir tane 32, bir tane 16 vardır, 8 ve 4 yoktur, sonra 2 ve 1 vardır. İkisi birbirinden farklıdır: Birinde 3 sayısal olarak, diğerinde karakter rakam olarak saklanmıştır.

Aynı ayrım daha önce 123 için de yapıldı. Üç karakterli "123" ile bir değişkende saklanan 123 sayısı farklıdır. Birincisinde üç karakter, ikincisinde tek bir sayı vardır.

10.2 333 Kaç Byte Kaplar?

Hoca 333'ün karakter olarak ve sayı olarak ne kadar yer kaplayacağını sordu.

Bir öğrenci sayısal olarak 16 bit, karakter olarak 24 bit dedi. Hoca "tam olarak böyle diyeme-yeceğim, doğru kısmı da var yanlış kısmı da" dedi ve başka cevaplar istedi.

> **Ders sonrası not:** Altyazıya göre bu öğrencinin cevabı, hocanın birazdan vardığı sonuçla aynıdır: 16 bit = 2 byte, 24 bit = 3 byte. Sayısal olarak cevap doğrudur.

Karakter olarak: Her rakam 1 byte'lık alan kaplar. Üç rakam olduğu için $3 \times 1 = 3$ byte gerekir.

Sayı olarak: "9 byte" ve "9 bit" cevapları geldi, ikisi de yanlıştı. Doğru cevap şöyle bulundu: 1 byte'a en fazla 255 yazılabilir. 333 buna sığmaz, bu yüzden 2 byte'lık alan gerekir. "9 bit" cevabının nereden geldiği şöyle görülebilir (ara adım eklendi):

$$333 = 101001101_2 \quad (9 \text{ bit})$$

Hoca neden 9 bit olmadığını da açıkladı: Belleği kullanırken 9, 10, 11 ya da 12 bit gibi alanlar kullanma şansınız yoktur. Alanlar 8'in katlarıdır: 8, 16, 24 ya da 32 bit. 1 byte'a sığmıyorsa 2 byte'ta saklanır.

> **Ders sonrası not:** 2 byte cevabını veren öğrenci "8 byte'a sığdıramadığımız için 16 byte" dedi. Doğrusu 8 bit ve 16 bittir.

10.3 4782 Kaç Byte Kaplar?

Hoca sayıyı büyüttü: 333 yerine 4782.

Karakter olarak: Dört rakam, karakter karakter $4 \times 1 = 4$ byte.

Sayı olarak: Bir öğrenci 2 byte dedi, çünkü sayı 2^{12}'den biraz büyüktü ($2^{12} = 4096$, ara adım eklendi). Peki 2 byte'a en fazla ne sığar? Hoca önce 2^{16}'yı hatırlattı: $2^{16} = 65536$. Sayı işaretli olsaydı 2 byte'lık alanda 32.768 civarına kadar gidilebilirdi. Hoca bunu kabaca hesaplamanın yeterli olduğunu söyledi. 4782 bu sınırların ikisinin de altındadır, yani 2 byte'a rahatça sığar.

> **Ders sonrası not:** 65536, 2 byte'ın ifade edebildiği farklı değer sayısıdır. Sıfır da sayıldığı için işaretsiz en büyük değer $2^{16} - 1 = 65535$'tir. İşaretli durumda, ikiye tümleyende en büyük değer $2^{15} - 1 = 32767$, en küçük değer -32768'dir.

Demek ki 4782 sayısal olarak 2 byte, rakam rakam saklanırsa 4 byte kaplar.

10.4 Metin İçindeki Sayılar

Hoca sonucu şöyle bağladı: Özel bir sebep yoksa sayılar sayı biçiminde saklanmalıdır. Peki metin içinde sayı geçiyorsa nasıl saklanır? Bir öğrenci harf harf saklandığını söyledi. Hoca "harf" demek yerine **karakter** denmesini istedi, çünkü harfler sembollerin yalnızca bir kısmıdır.

Metin içinde geçen 4782 ya da 333 gibi bütün sayısal ifadeler karakterler üzerinden saklanır. Ama program yazarken sayısal ifadeleri ayrıca kullanacaksak onları sayı tipindeki değişkenlerde saklamayı tercih ederiz, çünkü daha az yer kaplarlar.

11. Resim ve Video

Resimdeki en küçük birime **piksel** denir. Açılımı **picture element**'tir.

Hoca, resmi kağıt üzerinde göstermek için en uygun biçimi sordu. Cevap **matris**tir. Bir öğrencinin "kare" cevabını hoca kabul etmedi: Her resim kare olmak zorunda değildir. Videoda da 16x9 ve 4x3 gibi farklı biçimler vardır. Resmin yüksekliği (height) ve genişliği (width) ayrı ayrıdır. Matristeki her nokta bir piksele karşılık gelir.

Örnek: 1920x1080 resim. Her pikselin 1 byte ile ifade edildiğini varsayalım. Resim 1920×1080 byte yer kaplar. Sonuç hesaplanırsa (ara adım eklendi):

$$1920 \times 1080 = 2.073.600 \text{ byte}$$

Video. Bu tek bir resimdir. Video olsaydı ve saniyede 30 kare (30 frame per second) kullanılsaydı, bir saniyelik videonun kapladığı alan için 30 ile çarpmak gerekirdi (ara adım eklendi):

$$2.073.600 \times 30 = 62.208.000 \text{ byte}$$

Toplam videonun süresine göre kapladığı alan da bu şekilde kabaca hesaplanır. Hoca bunun kabataslak bir hesap olduğunu, çok farklı biçimler bulunduğunu, pikselde saklanan bilginin 1 byte'tan farklı olabileceğini ve bunların ileride konuşulacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Piksel başına 1 byte varsayımı büyük olasılıkla gri tonlu resimlere uyar. Renkli resimlerde her piksel için büyük olasılıkla kırmızı, yeşil ve mavi olmak üzere üç ayrı değer tutulur. Bu yüzden renkli resim aynı boyutta daha çok yer kaplar.

12. Ses, Diziler ve Değişkenler

Resim için matris kullanıldığı görüldü. Hoca ses bilgisinin nasıl bir yapıda saklandığını sordu. Sesi genelde **tek boyutlu** bir yapı olarak düşünebilirsiniz. Elbette sesin şiddeti ve farklı frekansları vardır, frekans (frequency) ve genlik (amplitude) gibi kavramlar devreye girer. Hoca bunların ayrıntısına girmedi. Genel yapı, sesin şiddetinin 2.3, 3.2 gibi değerler alarak saklanmasıdır. Resim genel olarak daha fazla yer kaplar.

Bu yapıya dizi (array) denir. İleride görüleceği gibi aynı tipteki verileri bellekte kesintisiz bir şekilde bir arada saklamak dizi kullanarak mümkün olur. Verileri saklamanın çok farklı yöntemleri de vardır, ama bellekte genelde bunlar kullanılır.

Tek bir değer saklamak istediğinizde ise bellekte tek bir alan kullanılır. Örneğin 5 değeri bir değişkenin (variable) alanına yazılır.

13. Veri Sıkıştırma

13.1 Neden Sıkıştırırız?

Veriler olduğu gibi saklanırsa bilgisayarda çok yer kaplar. Bu yüzden dosya sistemleri büyük verileri sıkıştırarak saklar ve siz eriştiğinizde çok hızlı bir şekilde açarak size sunar. Uzun süre kullanmadığınız birçok bilgi, siz söylemeden arka planda, dosya yönetim sistemi (file management system) tarafından sıkıştırılır. Çok sık eriştikleriniz belki sıkıştırılmaz, daha az eriştikleriniz sıkıştırılır.

Bir öğrenci sıkıştırmayı şöyle tanımladı ve hoca onayladı: Veriyi daha az yer kaplayarak saklamak.

13.2 Lossless ve Lossy

İki tür sıkıştırma vardır:

- **Kayıpsız (lossless):** Veri kaybetmeden sıkıştırma. Veri birebir olduğu gibi tekrar açılabilir. Hocanın örneği: 100 byte yer kaplayan bir veriyi 10 byte'a düşürdünüz. Tekrar 100 byte'a açtığınızda birebir aynı veriyi elde edersiniz.
- **Kayıplı (lossy):** Belirli oranda veri kaybederek sıkıştırma. Açtığınızda maalesef birebir aynısını elde edemezsiniz.

Kayıplıya örnek olarak bir öğrenci ses kayıtlarında duyulmayan frekansları gösterdi. Hoca çok sık kullanılan bir biçimi verdi: **MP3**. MP3, duymadığınız sesleri kodlamaz ve saklamaz. Özellikle yüksek seslerin ardından gelen düşük sesleri insan kulağının ayırt etmesi mümkün değildir. Görüntüde de aynı durum vardır: İnsan gözü bazı renk tonlarının farklılıklarını o hassasiyette görmez.

Bu yüzden kayıplı sıkıştırma daha çok ses ve görüntüde kullanılır. Her zaman böyle değildir, ama ağırlıklı olarak böyledir. Videolarda da çok sık karşımıza çıkar, çünkü çok yer kaplarlar.

> **Ek bilgi (derste anlatılmadı):** Derste kayıpsız sıkıştırma için belirli bir biçim adı verilmedi. Büyük olasılıkla ZIP ve PNG kayıpsız, JPEG ise MP3 gibi kayıplı biçimlere örnektir. Metin ve program dosyalarında tek bir bitin değişmesi bile içeriği bozabileceği için bu dosyalarda büyük olasılıkla kayıpsız yöntemler kullanılır.

Dersin Toparlaması

Dersin ilk yarısında kayan nokta gösterimi üç örnekle pekiştirildi. Sıra hep aynıdır: işaret biti, normal ikilik yazım, ilk 1'den başlayan mantis, excess-4 ile üs ve en sonda sağlama. $2\frac{3}{4}$ sayısı `0 | 110 | 1011`, $\frac{3}{8}$ sayısı `0 | 011 | 1100` olarak tam saklanır. $2\frac{5}{8}$ ise 5 bit gerektirdiği için `0 | 110 | 1010` olur ve $2\frac{4}{8}$ olarak işleme girer. Bu kesme hatası hiçbir hata mesajı üretmez. Gerçek bilgisayarlar bu sorunu 32 bitlik single precision ve 64 bitlik double precision biçimleriyle azaltır.

İkinci yarıda veri ile bilgi ayrıldı ve bilgisayarda saklanan veri türleri gözden geçirildi. Karakterler ASCII ve UTF-8 tablolarıyla kodlanır. Değerler ezberlenmez, karakter sabitleri ve aralarındaki farklar kullanılır. Bir sayının karakter olarak ve sayı olarak kapladığı alan farklıdır: 333 için 3 byte ve 2 byte, 4782 için 4 byte ve 2 byte. Resimler piksellerden oluşan matrislerle, sesler tek boyutlu dizilerle saklanır. Bu veriler çok yer kapladığı için kayıpsız ya da kayıplı yöntemlerle sıkıştırılır. Hoca sıkıştırma konusunun bir sonraki derste konuşulacağını, ardından kısa bir konudan sonra algoritmanın ne olduğu, nasıl tasarlandığı ve nasıl ifade edildiği konularına geçileceğini söyledi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.